

Pucho.ai

Pucho Pulse

Feature reference & operating guide

VERSION	1.0 · 18 Aug 2026
AUDIENCE	Founders · Sales · Solution Architecture · Developers
COVERS	Every feature, how to use it, and how to connect it to the production database

Pucho Toh Sahi!

Contents

- 1 What Pulse is for**
The three questions it exists to answer

- 2 The daily loop**
Who opens what, and when

- 3 Running a workshop end to end**
Create → deliver → enter attendees → follow up

- 4 Every view, explained**
All ten screens and the decisions each one supports

- 5 The propensity score (PPS)**
How an account gets a band, and what to do about it

- 6 Nudges, SLAs and the alert queue**
The ten triggers and how they are worked

- 7 Integrating with Pucho**
The two webhook contracts

- 8 Connecting to the production database**
Preflight, migrate, and the three things to know

- 9 Running the software**
Local setup, jobs, deployment, configuration

- 10 Reading the numbers correctly**
Definitions, red lines, and known limits

What Pulse is for

Pulse exists to answer three questions that were previously answered by opinion.

1 · Are the workshops getting clients?

Sixty workshops a month, roughly 1,200 free accounts, about ₹3.6L of credit given away. Pulse traces every account back to the workshop that produced it, so you can see which segment, which campaign and which partner turned attendance into paying customers — and which turned it into nothing.

Where: the headline sentence on *Today*, the workshop funnel on *Workshops*, and the segment scoreboard's KILL / DOUBLE / HOLD call.

2 · Is the credit grant being used, and is it working?

The 1,000-credit grant is the single biggest GTM cost. Pulse measures it on four dimensions: how deep accounts go, how fast they get there, what they spend it on, and — the one that matters — whether deeper usage actually converts.

Where: *Credit Grant*, especially the money chart and the cohort burn-down curve.

3 · Which account will buy next?

Every grant account is scored nightly from its own behaviour and placed in a band with a named next action. Partner and Sales time goes to Band A; Band D gets an automated drip and nobody's afternoon.

Where: the PPS leaderboard on *Search & 360* and the queue on *Today*.

The principle underneath all three. The score allocates attention; humans convert. Pulse never claims certainty about an individual account — it claims that if you spend your calling hours top-down through the band list, you will close more than if you spend them in signup order.

The daily loop

Pulse has one screen you *run* and nine you *consult*. Start at Today.

WHO	WHEN	OPENS	DOES
Founder	Every morning, on a phone	Today	Reads the headline sentence, checks what climbed into Band A and what fell out, glances at SLA breaches. Two minutes.
Founder	Monday	Workshops	Segment scoreboard. Kills any segment under 1% conversion after four workshops; doubles down on anything over 3%.
Pucho Sales	Every morning	Today → the queue	Works the queue top-down: breaches first, then the SLA clock. Each row opens the account's 360 and its close kit.
Solution Architecture	Before a workshop	Workshops → New workshop	Creates the workshop record: date, segment, campaign tag, partner, invited count, workbook link.
Solution Architecture	Same day, after delivery	Workshops → the workshop	Enters attendance, then pastes the attendee list. Each attendee becomes an account with its 1,000-credit grant.
Ops	Weekly	Inputs	Clears data gaps — partners missing a WhatsApp number or language, orgs missing industry, plans missing a credit quota.
Channel partners	As alerts fire	WhatsApp only	Receive hot-lead alerts, nudges and the Monday digest. No login — by design.

If **attendance is never entered**, the workshop shows a red row and its funnel understates itself permanently. The 18:00 reminder job exists for this, and unfilled workshops appear on Today under “Yesterday's workshops” until someone fixes them.

Running a workshop end to end

This is the core operating procedure. Everything else in Pulse is downstream of it.

1 Create the workshop — before the session

Workshops → *+ New workshop*. Six fields, all but one a dropdown: date & time, segment, campaign tag, channel partner, invited count, workbook link. Under two minutes.

The **campaign tag** is what lets you compare SEP26-WEB-CA against SEP26-WEB-TEX later. Skipping it is the most common way to lose an answer you will want in October.

2 Deliver the session

Nothing to do in Pulse. Collect attendee details however you already do — a sheet, a form, a notebook. You need six things per attendee: company, name, mobile, email, industry, company size.

Industry and company size are **required** because they are the firmographic input to the propensity score. An account missing them scores 0 of 8 on that component and ranks below identical accounts with complete profiles.

3 Enter attendance — same day

Open the workshop and type the attended count. This marks it DELIVERED and clears the red row.

4 Add the attendees — the primary path today

On the workshop page, either **Add attendee** (one form, for a walk-in) or **Paste a list** — comma or tab separated, one attendee per line, straight out of Excel:

```
Acme Traders, Ramesh Patel, 9812345678, ramesh@acme.in, Chemicals, 51-200
Shah Textiles, Nikhil Shah, 9898765432, nikhil@shah.in, Textiles, 11-50
```

Each row creates an Organization with full attribution, a User, and the 1,000-credit grant wallet. Rows are processed independently: a bad line reports itself by row and column ("*attendees.3.phone: Enter a valid Indian mobile number*") and the rest still land.

Pasting the same person twice is safe. Provisioning is idempotent per phone + workshop, so a duplicate returns the existing account rather than minting a second grant.

5 Watch it the next morning

Today → "Yesterday's workshops" shows whether accounts appeared. Then the workshop's own page shows the funnel with drop-off at every stage: invited → attended → accounts → activated in 7 days → engaged ≥300 credits → converted.

6 Follow up — driven by the queue, not by memory

From here Pulse takes over. The scan jobs raise alerts as accounts cross thresholds, go quiet, or sit at zero. Sales and partners work the queue; nobody has to remember to check.

Every view, explained

Ten screens. The first is for running the business; the rest are for understanding it.

Today THE ACTION QUEUE

The only screen that tells you what to *do*. Opens with a sentence — “2 of 4 delivered workshops have produced a paying client · 3 conversions at ₹902 credit cost each · every grant rupee has returned ₹5.54”.

- **Four tiles:** climbed into Band A this week, fell out of Band A, SLA breaches, grants expiring.
- **The queue:** every open alert, breaches first then by SLA clock. Each row shows the account, why it fired, the partner, and a countdown chip. Tapping a row opens that account's 360.
- **Yesterday's workshops:** did they produce accounts, and is attendance still missing?
- **Grants expiring:** credits still unused with under 14 days left — the last window a call can change.
- **Is the score working?:** conversion rate per band. Band A must beat B must beat C. A flat line here means stop trusting the bands.

Command Center EXECUTIVE OVERVIEW

MRR, credit burn with a 7-day moving average, activation over the last four weeks, DAU/MAU with stickiness, hot account count, failed payments. Below: daily burn, where credits go, signups vs activated, September GTM economics, and a needs-attention list. Every row links to the account.

Workshops · Workshop detail ATTRIBUTION

The list shows every workshop with its full funnel; on a phone each collapses to a card with the four numbers that matter. Below: campaign performance and the segment scoreboard with its KILL / DOUBLE / HOLD call at the <1%-after-4-workshops line.

The detail page is where a workshop is worked: stage funnel with carry-through percentages, attendee entry, and every account it produced with its band, score meter and current state.

Credit Grant IS THE ₹3.6L WORKING?

- **The money chart** — conversion rate by usage band. This curve must rise steeply. Flat means the grant is not driving conversion and the whole investment needs re-examining.
- **Burn-down by cohort** — average cumulative credits against days since signup, one line per signup month. Steeper is better; a cohort flattening under 300 never landed.
- **Burn milestones** — median days to 100 / 300 / 700 / 1,000, against their targets.
- **Is 1,000 the right number?** — credits already consumed at the moment of conversion, p25 / median / p75. These percentiles drive the October A/B on grant size.
- **Zero-utilisation aging** — accounts that never used a credit, bucketed 0–7 / 7–14 / 14–30 / 30+ days, each with an owner and an action.

Search & 360 ONE ACCOUNT, EVERYTHING

One box across partners, organizations and end users. Selecting a result opens the matching 360:

- **Partner 360** — health score with its four components, portfolio stats, three-month trend, and that partner's accounts only.
- **Org 360** — plan, partner, PPS, grant usage, a plain-language signal line, and the score trajectory as a sparkline.
- **End-user 360** — profile against behaviour, with an inline action note for zero-use.

Below sits the **PPS leaderboard**: every grant account with credits, Office days, apps, chats, momentum arrow, score meter, band and next action. Filter by band.

Inputs THE DATA THAT BREAKS THINGS

Two jobs. First, edit what production has no column for: each partner's WhatsApp language, number override and digest on/off. Second, a gap list where every gap names its consequence — orgs missing industry score 0 of 8 firmographic points; plans missing a credit quota cause the wrong plan to be pitched; workshops missing a workbook send a broken nudge link.

Credits & Revenue · Users & Engagement · Feature Usage · Partners & Funnel

The analytics drill-downs: burn and MRR and wallet utilisation; DAU/WAU/MAU, retention cohorts, power users, dormant seats; chat types, Pro Search and Deep Research, voice minutes, workflow leaderboard; partner health table, scorecard, SDR funnel in lifecycle order, contact-sales pipeline.

Close kit /KIT/:ORGID

One page a partner reads before dialling, generated from the account's own usage: the usage story, a conservative hours-saved estimate, the right plan computed from burn rate, the urgency line, and the 7-day bonus-carry offer.

The propensity score

Every grant account is scored 0–100 each night from behaviour alone, plus one firmographic prior. September sells Pucho Office, so Office behaviour *is* the score.

COMPONENT	POINTS	FULL MARKS AT	WHY IT IS IN THE SCORE
Office habit	25	~6 Office days in 14	Habit in daily documents is what renews
Credit depth	20	700 credits	Value, measured in the billing unit
Office app breadth	12	Excel + Word + PPT	A second app means a suite, not a trick
Office volume	10	50 Office chats	Raw usage weight
Embedding	15	own docs + 2 users + a workflow	Sunk cost and team adoption
Momentum	10	this week ≥ last week	Propensity is perishable
Firmographic	8	target industry × 51–500 people	Prior, replaced by live rates monthly
Recency penalty	–20...0	—	Silent 7 days –10; silent 14 days –20

Bands and what each one means

BAND	SCORE	ACTION	WHO SPENDS TIME
A	≥ 70	Close within 72 hours — send the close kit	Partner + Sales
B	45–69	Push to the aha event	Partner
C	20–44	Re-onboard: 15 minutes on their own data	Partner, low priority
D	< 20	Automated drip only	Nobody
W	override	Office re-demo on their own files — never a close attempt	Partner

Band W is the one people get wrong. It fires when an account has zero Office chats but ten or more general chats. They are busy in Pucho and absent from the product the workshop sold them. Pitching them a plan converts nothing; re-demoing Office on their own files converts some. W overrides the numeric score entirely — a W account never appears on a close list.

Two honest limits. A score is meaningless before day 3 — the leaderboard labels those accounts “new — score not yet meaningful”. And the whole scorecard is hand-weighted v0.1: the “Is the score working?” card on Today is how you check it against reality each month. If Band A stops out-converting Band D, adjust the weights in `config/scoring.ts` and re-run the snapshot.

Nudges, SLAs and the queue

Ten triggers, in priority order. Each fires once per account, is ledgered, and — where it carries an SLA — escalates if nobody acknowledges it.

#	TRIGGER	FIRES WHEN	SLA
1	Band A	Enters Band A or crosses 700 credits	72h → Sales, partner CC'd
2	Exhausted	Grant hits 1,000 — value proven, urgency peak	24h
3	No Office at 72h	72h post-workshop, zero Office chats	next digest
4	Momentum break	Office-active account silent 5 days	48h
5	Wrong lane	Band W detected	re-demo within 7d
6	Excel only	Strong Excel, no Word/PPT, day 14	—
7	Single player	>400 credits, still one user, day 14	—
8	Zero use 7d / 14d	No credits at 7 or 14 days	14d also alerts Sales
9	Weekly digest	Monday 09:00 per partner	—
10	Monthly scorecard	1st of the month, partner rank + health	—

The four safeguards

Dedupe

Enforced by a database index, not by application logic. Two scan workers racing on the same threshold crossing still produce exactly one alert.

Quiet hours

Nothing sends to a partner before 08:00 or after 21:00 IST. Real-time triggers queue and go out when the window opens.

Kill switches

Every trigger has an enable flag in `config/scoring.ts`. A bad template can be stopped without a deploy.

SLA escalation

An unacknowledged alert past its SLA escalates to Sales with the partner copied, and is marked so it never escalates twice.

Acknowledging

When a partner replies  on WhatsApp, Pucho posts the `ackUrl` that came with the alert. That stops the clock. Internal users can acknowledge from the queue. Acknowledging twice returns a conflict rather than silently doing nothing, so a double-tap is visible rather than misleading.

Integrating with Pucho

Two webhooks. Pucho owns registration capture and message delivery; Pulse owns attribution, scoring and the decision of who to contact.

Inbound — registrations arrive from Pucho

When Pucho's QR / web-workshop system captures a registration, it POSTs here and Pulse provisions the account with full attribution and the 1,000-credit grant.

```
POST /api/webhooks/registration
x-pulse-webhook-secret: $PULSE_INBOUND_WEBHOOK_SECRET
Content-Type: application/json

{
  "workshopId": "cmsy0yb...",          // OR "registrationToken": "CAWORKSHOP"
  "attendee": {
    "companyName": "Acme Traders",
    "fullName": "Ramesh Patel",
    "phone": "9812345678",           // +91 accepted
    "email": "ramesh@acme.in",
    "industry": "Chemicals",        // must match the managed list
    "companySize": "51-200"         // must match the managed list
  }
}
```

RESPONSE	MEANING
201 {created:true, organizationId}	New account provisioned with its grant
200 {created:false, organizationId}	That phone already registered for that workshop — safe retry
401	Bad or missing secret
404	Unknown workshop or registration token
422	Validation failed — the body names the exact field

Retries are safe. Provisioning is idempotent per phone + workshop, so a duplicate delivery returns 200 with the original organizationId and never mints a second grant. The managed industry and company-size lists are served by [GET /api/workshops/:id/attendees](#).

Outbound — nudges leave for Pucho to send

Pulse decides who, when, in which language and with which template; Pucho delivers. The payload includes the fully rendered text, so Pucho can send it directly even before a template is registered on its side.

```
POST $PULSE_OUTBOUND_WEBHOOK_URL
x-pulse-webhook-secret: $PULSE_OUTBOUND_WEBHOOK_SECRET

{
  "to": "9812300001",
  "template": "T1",
  "language": "gu",
  "params": { "org_name": "Seven Hundred Mills", "pps": 92, ... },
  "text": "🔥 HOT LEAD — Seven Hundred Mills\n Score 92/100. ...",
  "alertId": "cmsy1y3ej86vodf5201uzdpnu",
}
```

```
"ackUrl": "https://pulse.pucho.ai/api/alerts/cmsy1y3.../ack"
}
```

Respond **2xx**. Optionally return `{"id": "<message-id>"}` and Pulse stores it against the alert for tracing. When the partner replies , POST the **ackUrl** with the same secret header.

Leave **PULSE_OUTBOUND_WEBHOOK_URL** unset and every send becomes a dry run that logs the exact payload. Dedupe, quiet hours, kill switches, the SLA clock and the ledger all still run, so the entire notification engine can be tested and demonstrated before Pucho's sender exists.

Language and number do not come from the production schema. **ChannelPartner** has no language column and its phone number is often a landline, so both live in Pulse's own **PulsePartnerSettings** table and are edited on the Inputs screen. A partner with neither a number nor an override appears there as "no send route".

Connecting to the production database

Pulse was built and verified against your 17 Aug 2026 schema dump. This is the sequence to connect it to the live database safely.

1 Run the preflight — read-only, changes nothing

```
export DATABASE_URL='postgres://...' # primary - writes
export REPLICA_URL='postgres://...' # replica - every analytics read
npx tsx db/preflight.ts --replica
```

It checks every table, column and enum value Pulse depends on; reports whether the objects it wants to create already exist; confirms the Office-signal join actually resolves on your data; tells you which grant scope to configure; and warns about Prisma drift. It exits non-zero on any blocking mismatch, so it can gate a deploy.

2 Add the models to Prisma — before migrating

Your database is Prisma-managed. Pulse's tables are created in raw SQL, so Prisma will treat them as drift and generate a **DROP** unless it knows about them. Paste `prisma/pulse-models.prisma` into your `schema.prisma` (it also lists the three fields to add to your existing Organization model), then:

```
npx prisma migrate dev --create-only
```

Prisma should report **no changes**. If it wants to create or drop anything, stop — the database and the fragment have diverged. Constraint and index names in the migration deliberately follow Prisma's own convention so this comes out clean.

3 Migrate

```
npm run db:migrate
```

Applies `db/migrations/*.sql` in order, tracked in a `PulseMigration` table. Idempotent — re-running is a no-op.

Exactly what changes

OBJECT	TYPE	NOTE
<code>Workshop</code> , <code>PropensityLog</code> , <code>GtmAlert</code> , <code>PulseJobRun</code> , <code>PulsePartnerSettings</code>	new tables	None of these names exist in your dump
<code>Organization."workshopId"</code> , <code>."signupSegment"</code> , <code>."signupSource"</code>	new columns	Nullable; all three names verified free
<code>Organization_workshopId_idx</code> , <code>Organization_channelPartnerId_idx</code>	new indexes	You had no index on <code>channelPartnerId</code>
FKs on the new tables, plus <code>Organization_workshopId_fkey</code>	new constraints	Matching your existing ON UPDATE CASCADE style

Nothing else is touched: no existing column altered, no existing index dropped, no data rewritten. `db/rollback.sql` removes exactly this set and nothing more.

Three things to know

1 • The **Organization** foreign key is created **NOT VALID**, on purpose. Validating it would take an **ACCESS EXCLUSIVE** lock and a full scan of a large live table. The column is brand new so every row is NULL and there is nothing to validate. Run this in a quiet window whenever convenient:

```
ALTER TABLE "Organization" VALIDATE CONSTRAINT "Organization_workshopId_fkey";
```

2 • Set the grant scope deliberately. **PULSE_GRANT_SCOPE=PARTNER_FALLBACK** counts any account with a channel partner; **=WORKSHOP** counts only accounts Pulse attributed. Start on the fallback, switch once enough accounts carry **signupSource='WORKSHOP'**. The preflight prints both counts.

3 • One index from the original spec was deliberately skipped. **DATA_MODEL.md** asks for an index on **FreeCreditUsage(organizationId)**. Your database already has **FreeCreditUsage_organizationId_userId_idx**, whose leftmost prefix serves every lookup Pulse makes. A second index would cost writes on a hot table and buy nothing.

The read/write split is enforced in code

Analytics reads can only reach **REPLICA_URL** — the read pool opens every connection with **default_transaction_read_only = on**. Writes can only reach **DATABASE_URL**, and are rejected unless the target is a table Pulse owns; on **Organization**, only the three attribution columns are writable. **DELETE** is refused outright. In production the app will not start without **REPLICA_URL**, so analytics can never silently fall back to the primary.

Running the software

Local setup — about two minutes

```
cp .env.example .env.local
npm install
createdb pulse_fixture
npm run db:fixture      # production-shaped schema + migrations + seed
npm run job all         # populate the Today queue
npm run dev             # → http://localhost:3100
npm test                # 75 tests
```

The fixture is a local stand-in for production, generated verbatim from your dump by `scripts/extract-fixture-schema.py`, seeded with accounts covering every band and bucket edge — 0, 99, 100, 299, 300, 699, 700, 999 and 1,000 credits, a wrong-lane account, a stale account, a multi-user account, an expired wallet and a partner with no phone number. It refuses to run against anything that does not look like a local database.

Scheduled jobs

JOB	SCHEDULE (IST)	DOES
<code>pps-snapshot</code>	02:00 nightly	Scores every grant account into PropensityLog
<code>hot-scan</code>	every 30 min, 08:00–21:00	Band-A entries, ≥700 crossings, exhaustion
<code>office-nudge</code>	hourly	72h post-workshop with zero Office use
<code>momentum-scan</code>	08:30 daily	Momentum break, wrong lane, Excel-only, single player, zero-use ladder
<code>partner-digest</code>	Monday 09:00	Weekly per-partner summary
<code>sla-escalation</code>	hourly	Unacknowledged past SLA → escalate to Sales
<code>attendance-reminder</code>	18:00 daily	Today's workshops with no attendance entered
<code>calibration-report</code>	1st, 03:00	Band monotonicity, quintiles, aha lift

Run one with `npm run job <name>`, all with `npm run job all`, or the whole cron table in a worker with `npx tsx jobs/schedule.ts`. Every run writes a `PulseJobRun` row with its outcome, so “did last night's snapshot actually run?” is a query, not a guess.

Configuration

VARIABLE	PURPOSE
<code>DATABASE_URL</code>	Primary. Writes only.
<code>REPLICA_URL</code>	Replica. Every analytics read. Required in production.
<code>PULSE_GRANT_SCOPE</code>	<code>WORKSHOP</code> or <code>PARTNER_FALLBACK</code>
<code>PULSE_INBOUND_WEBHOOK_SECRET</code>	Shared secret for registrations arriving from Pucho
<code>PULSE_OUTBOUND_WEBHOOK_URL</code> / <code>_SECRET</code>	Where nudges go. Unset = dry run.
<code>PULSE_SALES_PHONE</code>	Escalation target for SLA breaches
<code>CREDIT_VALUE_INR</code> · <code>GRANT_CREDITS</code>	₹0.30 and 1,000 — the GTM economics constants

VARIABLE	PURPOSE
<code>PULSE_BASE_URL</code>	Used in close-kit and ack links

Deploying

Node 20+. `npm ci && npm run build && npm start` on port 3100, plus one worker process running the scheduler. Roles come from an `x-pulse-role` header your reverse proxy sets from the Keycloak token — `pulse.admin`, `pulse.sales`, `pulse.workshop`. Every endpoint is guarded except the optional public registration page.

Reading the numbers correctly

Definitions worth agreeing on

TERM	MEANS EXACTLY
Grant account	Depends on PULSE_GRANT_SCOPE — either any account with a channel partner, or only accounts Pulse attributed to a workshop
Activated	Used at least one credit within 7 days of signup
Engaged	Consumed ≥ 300 grant credits
Hot	≥ 700 grant credits, or Band A
Converted	Has an active OrganizationPackage
Office signal	Chats of type PUCHO_OFFICE_EXCEL / WORD / POWER_POINT, joined to the org through OrganizationUser
Revenue per credit rupee	Collected revenue $\div$ (credits consumed $\times$ ₹0.30). Must cross 1.0.

Red lines

MEASURE	HEALTHY	ACT WHEN
Grant utilisation	50–75%	Below 50% — the grant is not landing
Exhaustion rate	20–40%	Below 20% — accounts never reach the close moment
Never landed (<100 credits)	under 35%	Above 35% — the workshop is not manufacturing first use
Median days to 300	≤ 14	Slower means the aha moment is not happening in the room
Segment conversion	$\geq 3\%$	Below 1% after 4 workshops — kill the segment
Partner SLA compliance	$\geq 90\%$	Below — alerts are being ignored, review the partner

What Pulse does not do

No customer-facing dashboards. No partner web portal — partners are WhatsApp-only by design. No machine learning: v0.1 is a hand-weighted scorecard, and PropensityLog is accumulating the history that a model would eventually need. No billing changes of any kind; every billing table is read-only to Pulse.

Known gaps in this build

GAP	IMPACT	WHAT REMAINS
Materialized views	Queries run live; fine at current scale	Five views + refresh jobs when the grant population grows
Close-kit PDF	Kit renders as a web page	HTML $\rightarrow$ PDF step and a signed URL
Keycloak	RBAC enforced, role from a proxy header	Verify the token directly — one function
OTP delivery	Only affects the optional self-service page	Point it at an SMS provider
Grant provisioning	Pulse writes the wallet directly	Call your existing provisioning service instead

How to trust a number on screen. Every figure traces to a query in `docs/SQL_LIBRARY.sql` , and 41 automated tests run the library's text and the application's version side by side against a seeded database, requiring identical results. The propensity scorecard is implemented twice — once in SQL, once in TypeScript from `config/scoring.ts` — and a test requires the two to agree on every account. If someone changes a weight in one place and not the other, the build fails rather than the dashboard quietly lying.

Pucho Pulse · Feature reference & operating guide · Version 1.0 · 18 August 2026

Source: [pucho-pulse-source.zip](#) · README.md carries the developer quick-start · [docs/](#) carries the original product specification.

Pucho Toh Sahi!